Northwind Logistics — Shipment Exception Desk

The problem

Northwind Logistics handles shipments for dozens of retail clients. When something goes wrong with one -- delayed, damaged, or lost in transit -- a customer files an exception report, and right now a human ops analyst reads every single one: decide the category, work out compensation per policy, draft a reply, and escalate anything high-value or unclear to a manager. It's slow, and different analysts don't always apply the compensation rules the same way.

Your task is to build the **Shipment Exception Desk**: an automated triage system, built with **LangChain**, that reads an incoming exception report, classifies it, calculates compensation, and either resolves it automatically or routes it to a human -- with a working interface an analyst can actually use.

This is your first hands-on LangChain project. You'll use the same `prompt | model | parser` chain pattern from Class 2 to call an LLM for classification and message-drafting, and plain Python (`if`/`elif`, function calls) to wire those chains together with rule-based business logic. There's no new framework beyond what you've already learned -- it does **not** use RAG, AWS, MCP, or any graph-orchestration library.

The asks

Ask 1 — Build the exception-processing pipeline.

`chains.py` gives you three ready-made LangChain chains: `classify_chain` (classifies a report as `delayed` / `damaged` / `lost` / `unknown`), `escalate_chain` (drafts an internal note for a manager), and `draft_email_chain` (drafts a customer-facing email). Your job is `tools.py`'s three compensation calculators -- each returning a structured dict, not a formatted string, since the compensation amount drives the next decision -- and `pipeline.py`'s `process_exception(...)`: call `classify_chain`, route to the right compensation function, decide whether to escalate (compensation over a threshold that's *lower for premium customers* than standard ones, or automatically for anything `unknown`), and call the right drafting chain.

Ask 2 — Real daily aggregation.

Every exception processed in a session should feed a Daily Triage Summary in `session.py` that does real aggregation, not a flat recap: total compensation paid, escalation rate, and the **costliest category** by total compensation (a category with many small payouts can beat one with a single large one).

Run `triage_check.py` against your finished pipeline: it feeds four canned reports through it, each with a stated expected outcome (a mild delay, a high-value loss, a minor damage claim, and a garbled unclassifiable report). If it routes all four correctly -- especially the garbled one, which must escalate regardless of value -- your logic is working. If any land on the wrong branch, the bug is almost always in your threshold check or category branch, not the LLM call.

Ask 3 — A working Gradio interface (required).

Wrap the pipeline in a Gradio app (`app.py`): a form to submit a report (text, shipment value, customer tier), a panel showing the outcome and the steps it went through, a running Daily Triage Log, and a button to pull the Daily Summary. This is the interface you'll demo -- not optional, and not just a CLI print loop.

Suggested approach

1. Get `tools.py`'s three compensation functions working and tested standalone before touching `pipeline.py`.
2. Build just the classification + compensation half of `pipeline.py` first (classify -> compensate -> return, skip escalation for now) and get one report through it via `main.py`.
3. Add the escalation decision and the two drafting chains.
4. Add `session.py`'s aggregation, then run `triage_check.py` to prove the whole thing routes correctly end-to-end.
5. Only once `main.py` and `triage_check.py` both work should you move to `app.py` -- the UI should be the last layer on top of logic you already trust.

Milestones

1. Environment setup — venv, `pip install -r requirements.txt`, `.env` configured with your API key.
2. `calculate_delay_compensation` implemented and tested directly.
3. `calculate_damage_compensation` implemented and tested directly.
4. `calculate_lost_compensation` implemented and tested directly.
5. `pipeline.py`'s classification + compensation branch working, checked via `main.py` (report in, compensation out, escalation not wired yet).
6. The escalation decision and both drafting chains wired into `pipeline.py`.
7. `session.py`'s `generate_daily_summary` implemented with real aggregation (total compensation, escalation rate, named costliest category).
8. `python triage_check.py` passes — all four scenarios land on their stated expected outcome.
9. `app.py` built — the full flow (submit report → outcome + steps → Daily Triage Log → Daily Summary) works in the browser.

About the helper files

- **`.env`** / **`.env.example`** — holds your API key(s) (e.g. `GROQ_API_KEY`).
- **`.gitignore`** — tells git to ignore your virtual environment, `.env`, and Python cache files.

- **`requirements.txt`** — the exact list of Python packages this project needs (`langchain`, `langchain-openai`, `python-dotenv`, `gradio`).

Scope for improvement (take-home ideas)

- Add a fourth handler category (e.g. "wrong item shipped") end-to-end, including its own compensation rule.
- Persist the Daily Triage Log to a file so it survives restarting the app.
- Flag repeat exceptions from the same customer within a time window for extra scrutiny.
- Try swapping the fixed dollar thresholds for a percentage-of-shipment-value threshold instead, and compare the escalation rate it produces.
- Add a lightweight retry: if `classify_chain` returns something you don't recognize, call it a second time with a stronger prompt before falling back to `unknown`.

What "successful completion" looks like

Running `python app.py` should let you submit a shipment exception report in the browser and see it correctly classified, compensated per the rules, routed to either an auto-resolved customer email or a manager escalation, and logged into a Daily Triage Log with a working Daily Summary button. `python triage_check.py` should pass all four scenarios.